

CURSO DE PYTHON

**Corporación Ecuatoriana para el Desarrollo de la
Investigación y la Academia**

Ing. María Fernanda Granda J., PhD

20-07-2026

**“Conectando ideas,
transformando
sociedades”**

Tema 3: Diseño de Aplicaciones con Clases, Objetos, Encapsulamiento, Herencia y Polimorfismo

Objetivo de Aprendizaje:

Al finalizar la sesión, el estudiante será capaz de diseñar soluciones utilizando Programación Orientada a Objetos (POO), modelando entidades del mundo real mediante clases, objetos, herencia y polimorfismo para desarrollar software más mantenible, escalable y reutilizable.

Competencias:

- Comprender el paradigma orientado a objetos.
- Diseñar clases y objetos correctamente.
- Implementar encapsulamiento para proteger datos.
- Aplicar herencia para reutilizar código.
- Utilizar polimorfismo para crear sistemas flexibles.

- Modelar problemas reales mediante objetos.

Tema 3: Programación Orientada a Objetos (POO) en Python (3h)

- Introducción a POO
- Clases y Objetos
- Métodos y Constructores
- Encapsulamiento
- Herencia
- Polimorfismo
- Tarea

¿Por qué Programación Orientada a Objetos?

Actividad inicial

Revisar el siguiente código:

- ¿Qué pasa si tenemos 100 empleados?
- ¿Dónde ponemos las funciones relacionadas?
- ¿Cómo evitamos repetir código?

```
empleado1 = {  
    "nombre": "Juan",  
    "cargo": "Ingeniero",  
    "salario": 1200  
}
```

```
empleado2 = {  
    "nombre": "Ana",  
    "cargo": "Analista",  
    "salario": 1000  
}
```

Conceptos de la POO

La POO permite representar entidades reales como objetos.

Mundo Real

Estudiante

Vehículo

Producto

Cliente

Cuenta Bancaria

Clase

Estudiante

Vehiculo

Producto

Cliente

CuentaBancaria

Analogía

- **Clase:** Es el plano de una casa.
- **Objeto:** Es la casa construida

Clases y Objetos

Definición de Clase: es una plantilla o plano que define cómo se crearán los objetos.

Permite empaquetar datos (llamados atributos) y comportamientos (llamados métodos) en una sola estructura

```
# definición de clase
class <nombre_de_la_clase>(<super_clase>):

    def __init__(self, <params>):
        <expresion>

    def <nombre_del_metodo>(self, <params>):
        <expresion>
```

Clases y Objetos

Definición de Clase: es una plantilla o plano que define cómo se crearán los objetos.

Permite empaquetar datos (llamados atributos) y comportamientos (llamados métodos) en una sola estructura

Ejemplo sencillo:

```
class Laptop:
    """
    Clase empleada para trabajar con Laptops
    """
    has_ssd=True
```

Crear objetos:

```
laptop = Laptop()
```

Clases y Objetos

Método para inspeccionar una clase

- La función integrada dir() en Python devuelve una lista de los atributos, métodos y variables válidos asociados a cualquier objeto.
- Métodos especiales o métodos mágicos (dunder methods), cuyo nombre comienza y termina con doble guion bajo (__).

Ejemplo sencillo:

```
print(dir(laptop))
```

Salida:

```
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__firstlineno__', '__format__', '__ge__', '__getattr__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__static_attributes__', '__str__', '__subclasshook__', '__weakref__', 'has_ssd']
```


Clases y Objetos

Método	Descripción
<code>__class__</code>	Indica la clase a la que pertenece el objeto.
<code>__dict__</code>	Diccionario con los atributos del objeto.
<code>__doc__</code>	Devuelve la documentación (docstring) de la clase.
<code>__module__</code>	Nombre del módulo donde fue definida la clase.
<code>__init__</code>	Constructor de la clase. Se ejecuta al crear el objeto.
<code>__str__</code>	Define cómo se muestra el objeto al usar <code>print()</code> .
<code>__repr__</code>	Representación oficial del objeto, usada para depuración.
<code>__eq__</code>	Permite comparar objetos con <code>==</code> .
<code>__hash__</code>	Devuelve el valor hash del objeto.
<code>__sizeof__</code>	Indica el tamaño del objeto en memoria.
<code>__dir__</code>	Personaliza el resultado de <code>dir()</code> .
<code>__getattr__</code>	Se ejecuta cada vez que se accede a un atributo.
<code>__setattr__</code>	Se ejecuta cuando se asigna un valor a un atributo.

Clases y Objetos

- Todas las clases en Python heredan de la clase object.
- El objeto también dispone automáticamente de muchos métodos heredados.

Método	Descripción
<u>__delattr__</u>	Se ejecuta al eliminar un atributo.
<u>__new__</u>	Crea el objeto antes de que se ejecute <u>__init__</u> .
<u>__reduce__</u> , <u>__reduce_ex</u>	Utilizados para <u>serialización (pickle)</u> .
<u>__format__</u>	Controla el formato del objeto con <u>format()</u> .
<u>__gt__</u>	Comparación "mayor que" (>).
<u>__ge__</u>	Comparación "mayor o igual" (>=).
<u>__lt__</u>	Comparación "menor que" (<).
<u>__le__</u>	Comparación "menor o igual" (<=).
<u>__ne__</u>	Comparación "distinto de" (!=).
<u>__weakref__</u>	Soporte para referencias débiles (<u>weak references</u>).
<u>__subclasshook__</u>	Personaliza la comprobación de subclases.

Clases y Objetos

Método Constructor `__init__()`

- Si no tiene método constructor, Python genera uno por defecto
- Es el primero que se ejecuta automáticamente cuando inicializamos una clase.
- Puede recibir valores para los atributos de cada objeto.
- **Atributo** son las variables que definen a los objetos de una clase

```
# definición de clase
class <nombre_de_la_clase>(<super_clase>):

    def __init__(self, <params>):
        <expresion>

    def <nombre_del_metodo>(self, <params>):
        <expresion>
```

```
class Laptop():
    """
    Clase empleada para trabajar con Laptops
    """
    def __init__(self, marca, procesador, memoria, ssd):
        self.marca = marca
        self.procesador = procesador
        self.memoria = memoria
        self.has_ssd = ssd
```

Clases y Objetos

Instanciar un objeto

- La clase es un molde, a los objetos creados se les conoce como instancias.
- Cuando se crea una instancia, se ejecuta el método `__init__`
- Todos los métodos de una clase reciben implícitamente como primer parámetro `self`
- Instanciación es cuando el objeto se crea en la memoria mientras se ejecuta el código

```
if __name__ == '__main__':  
    #Instanciando el objeto  
    laptop1 = Laptop('Dell', 'Core-i7', 16, True)
```

Clases y Objetos

¿Para qué sirve `if __name__ == "__main__":`?

Python solo importa la clase, sin ejecutar el código de prueba que puede estar incluido en la clase que se importa.

¿Por qué usar una función `main()`?

Porque separa la lógica del programa de las definiciones de clases y funciones, haciendo el código más organizado, reutilizable y fácil de mantener. Este es el estilo que encontrarás en la mayoría de proyectos profesionales en Python

```
class Laptop:

    def __init__(self, marca, memoria):
        self.marca = marca
        self.memoria = memoria

    def mostrar(self):
        print(f"Marca: {self.marca}")
        print(f"Memoria: {self.memoria}")

def main():
    laptop = Laptop("Lenovo", 16)
    laptop.mostrar()

if __name__ == "__main__":
    main()
```

Clases y Objetos

Métodos de una clase

- En Python siempre los métodos ubicados dentro de una clase comienza con un parámetro llamado self y luego los demás parámetros.
- Self (a sí mismo), es una variable útil para acceder a las variables del propio objeto

```
def <nombre_del_metodo>(self, <params>):  
    <expresion>
```

Clases y Objetos

Atributos por defecto de una clase

- Se pueden definir en el constructor

```
#Método Costructor
def __init__(self, marca, procesador, memoria = 32, ssd = True):
    self.marca = marca
    self.procesador = procesador
    self.memoria = memoria
    self.has_ssd = ssd
```

- Probar y mostrar

```
#Instanciando el objeto
laptop1 = Laptop('Dell', 'Core-i7', 16, False)
```

```
#Instanciando el objeto
laptop1 = Laptop('Dell', 'Core-i7')
```


Clases y Objetos

El método `__dict__`

- Cuando se crea una clase y sus instancias, Python utiliza `__dict__` para gestionar los datos
- No es una función ni un método, sino un diccionario (o mapeo) integrado que almacena los atributos modificables y el espacio de nombres de un objeto

Diferencia entre Clases e Instancias:

- En las instancias, `__dict__` solo guarda las variables de instancia.
- En las clases, `__dict__` almacena los atributos de la clase y los métodos definidos en ella

```
if __name__ == '__main__':  
    laptop = Laptop("Lenovo", 'Core.i7', 16, True)  
    print(laptop.__dict__)
```

```
{'marca': 'Lenovo', 'procesador': 'Core.i7', 'memoria': 16, 'has_ssd': True}
```


Clases y Objetos

Diferencia entre Clases e Instancias:

- **Introspección y modificación:** Puedes utilizar `__dict__` para inspeccionar o incluso agregar, modificar y eliminar atributos dinámicamente de forma directa.

```
if __name__ == '__main__':  
    laptop = Laptop("Lenovo", 'Core.i7', 16, True)  
    laptop.__dict__['precio'] = '2000'  
    print(laptop.precio)  
    print(laptop.__dict__)
```

Clases y Objetos

Método destructor

- Elimina las instancias de una clase, es decir elimina el objeto
- `__del__()`
- Se invoca automáticamente cuando usamos el comando **del**

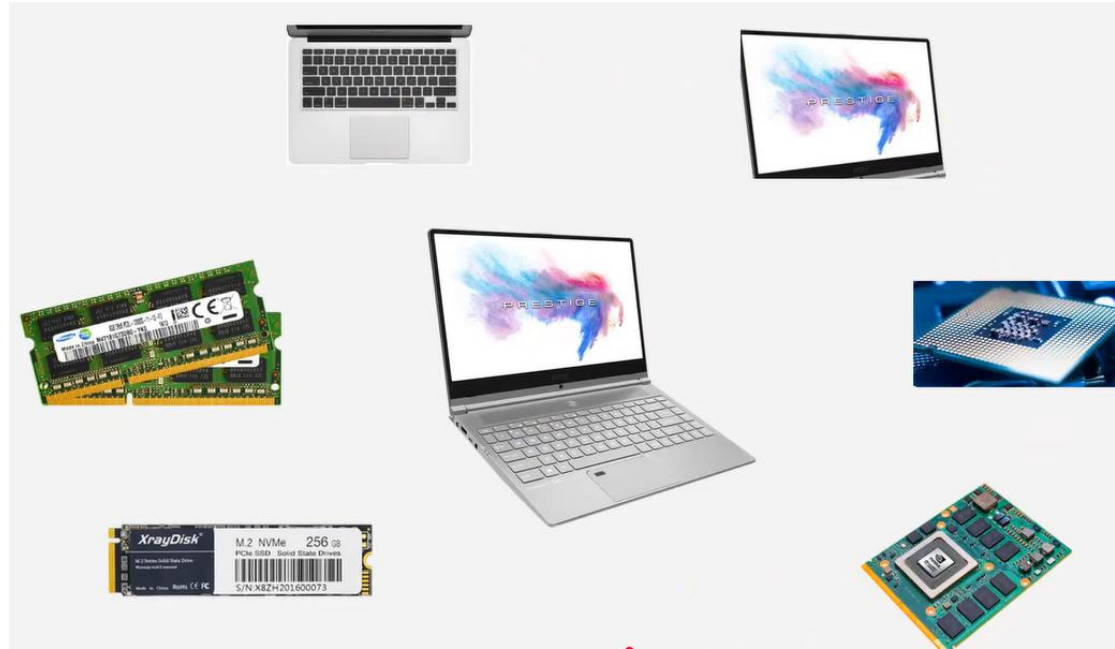
```
#Método Destructor
def __del__(self):
    print('El objeto fue destruido')
```

```
if __name__ == '__main__':
    laptop = Laptop("Lenovo", 'Core.i7', 16, True)
    laptop.__dict__['precio'] = '2000';
    print(laptop.precio)
    print(laptop.__dict__)

    del laptop
```

Descomposición

- La idea de la POO es poder dividir un problema en sub problemas mucho más pequeños
- Las clases permiten crear dentro de nuestro código mayores abstracciones en forma de componentes
- Cada clase entonces se va a encargar de una parte del problema por lo tanto el código se vuelve mucho más sencillo de leer.



Abstracción

- Permite enfocarnos únicamente en la información relevante del problema
- Separa la información central de los detalles secundarios
- Se pueden utilizar variables y métodos
- Ejemplo:
- **Información relevante:** Presionar el botón para el piso que quiere ir
- **Información no relevante** (abstracción)
 - Velocidad del motor
 - Movimiento de las poleas
 - Regulación de frecuencia en el variador



Métodos de una Clase

Existen 3 tipos de métodos

- Métodos de instancia: de cada objeto (acciones o funciones de cada uno=
- Métodos estáticos
- Métodos de clase

Métodos de Instancia de una Clase

- Toman el parámetro self como primer parámetro representando la instancia del método.
- Seguidamente puede tener más inputs.

```
class Cilindro:

    #Constructor
    def __init__(self, radio = 1, height = 1, color = 'green'):
        self.radio = radio
        self.height = height
        self.color = color

    #Métodos de Instancia (Parametro de entrada es propia instancia)
    def perimeter(self):
        return 2*self.radio + self.height

    #Método de Instancia
    def area(self):
        return 3.1416 * self.radio ** 2

    #Método de Instancia
    def volumen(self):
        return 3.1416 * self.radio ** 2 * self.height
```

```
if __name__ == '__main__':
    cilindro1 = Cilindro(2, 4, 'azul')
    print(f'El perimetro del cilindro es {cilindro1.perimeter()}')
    print(f'El area del cilindro es {cilindro1.area_base()}')
    print(f'El volumen del cilindro es {cilindro1.volumen()}')
```

```
print(f'El cilindro 1 es:\n{cilindro1}')
```

```
def __str__(self):
    return (f'Radio: {self.radio}\nHeight: {self.heighth}\nColor: {self.color}')
```

Métodos Estáticos de una Clase

- Los métodos estáticos en Python son extremadamente similares a los métodos de nivel de clase de Python (métodos de instancia), con la diferencia de que un método estático está vinculado a una clase en lugar de a los objeto de esa clase.
- Se puede llamar a un método estático sin un objeto para esa clase. Esto también significa que los métodos estáticos no pueden modificar el estado de un objeto ya que no están vinculados a él.
- Se definen usando **@staticmethod**, que se añade antes de definir el método estático respective.

Métodos Estáticos de una Clase

```
#Métodos estáticos
@staticmethod #decorador
def are_equal_height(cilindro1, cilindro2):
    if cilindro1.height == cilindro2.height:
        return True
    return False
```

```
if __name__ == '__main__':
    cil1 = Cilindro(3, 5, 'Rojo')
    cil2 = Cilindro(1, 2+3)

    print(f'¿Son los cilindros iguales? = {Cilindro.are_equal_height(cil1, cil2)}')
```


Métodos Estáticos de una Clase

- Los métodos estáticos tienen un caso de uso muy claro, cuando necesitamos alguna funcionalidad que no sea un Objeto sino la clase completa, hacemos un método estático.
- Es bastante ventajoso usarlos cuando necesitamos crear métodos de utilidad, ya que generalmente no están vinculados al ciclo de vida de un objeto.
- Para el método estático no se necesita pasar **self** como primer argumento.

Métodos de Clase

- Para usar un método de clase se emplea `@classmethod` para marcarlo como un método de clase.
- Toman como primer argumento el argumento **cls** que apunta a la clase, y no a la instancia del objeto, cuando se llama al método.
- Debido a que el método de clase solo tiene acceso a este argumento `cls`, no puede modificar el estado de la instancia del objeto (requeriría acceso a `self`). Sin embargo, los métodos de clase aún pueden modificar el estado de la clase que se aplica en todas las instancias de la clase.

Métodos de Instancia de una Clase

- Por ejemplo queremos generar un cilindro estándar.

```
#Método de Clase
@classmethod
def tank_cilinder(cls):
    radio = 2
    height = 3.5
    return cls(radio, height)
```

Se hace uso de una clase dentro de la misma clase para crear un nuevo objeto

```
cilindro3 = cilindro.tank_cilinder()
print(cilindro3)
```

Para practicar

Ejemplo sencillo:

```
class Estudiante:  
    pass
```

Crear objetos:

```
est1 = Estudiante()  
est2 = Estudiante()
```

Ejemplo 1

Agregar atributos

```
class Estudiante:  
  
    def __init__(self, nombre, carrera):  
  
        self.nombre = nombre  
        self.carrera = carrera
```

Crear instancias

```
est1 = Estudiante("María", "Software")  
  
print(est1.nombre)  
print(est1.carrera)
```

Ejemplo 2

Crear clase Libro

Atributos:

- Título
- Autor
- Paginas

Crear tres objetos.

Mostrar información.

```
libro1 = Libro(  
    "Python Avanzado",  
    "Juan Pérez",  
    350  
)
```

Ejemplo 3

- **Clase cuenta bancaria**

Implementar operaciones:

```
depositar()  
retirar()  
consultar_saldo()
```

Realizar un caso práctico

```
cuenta1 = CuentaBancaria("María", 500)
```

GRACIAS